For a practical example of how to use this hook in a form component, refer to the following interview question.

Scenario: In a frontend interview, Jane was tasked with integrating a form with the Web3Forms API (https://web3forms.com), a privacy-friendly service that lets developers receive form submissions via email without any backend setup. The requirements included using a standard HTML form with the action set to the Web3Forms endpoint, submitting via POST, and authenticating with a hidden `access_key` field.

Analysis: The interviewer is evaluating the candidate's ability to build a secure, user-friendly form that integrates with an external API like Web3Forms, with a focus on core skills such as correctly implementing form attributes (like `action`), securely managing sensitive data (such as API keys), and providing client-side validation. They also look for clear state management, effective user feedback, and modular, maintainable code. In a coding interview, where time constraints may prevent full implementation, it is essential for candidates to prioritize core functionality and clearly communicate which features are implemented, which are omitted, and the reasoning behind those choices. This approach demonstrates not only technical proficiency and attention to detail but also strong communication skills, a focus on user experience, and adherence to best practices—all of which significantly contribute to a positive evaluation.

Jane's Approach: She communicated her plan to the interviewer as follows:

- Set the `action` attribute to https://api.web3forms.com/submit.
- Use the POST method for form submission.
- Include a hidden input field named `access_key`, populated with the provided access key.
- Add input fields for user data, such as `name` and `email`.

Here are validation requirements:

- Restrict the `name` field to accept only letters and spaces.
- Restrict the `email` field to accept only valid email addresses.
- Ensure both `name` and `email` fields are required.